Semantic Search Applications, Retrieval-Augmented Generation (RAG), and Grounded Context Engineering

Created by Dr. David Gomillion with assistance from AI

Executive Summary & Core Concepts

Large Language Models (LLMs) possess vast internal knowledge learned during pre-training. However, for enterprise applications, standalone LLMs suffer from three critical limitations: **knowledge cutoff dates**, **lack of access to private domain data**, and **hallucinations** (generating plausible-sounding but factually incorrect assertions).

Retrieval-Augmented Generation (RAG) solves these limitations by combining information retrieval from external database systems (such as vector stores, relational databases, and knowledge graphs) with the generative reasoning powers of an LLM. Rather than relying on the LLM's static weights, a RAG system retrieves fresh, authoritative context chunks relevant to a user's prompt and injects them directly into the LLM's prompt window as grounded truth.

This reading explores the architecture of enterprise semantic search applications, the multi-stage RAG pipeline, prompt engineering for grounded context, hallucination mitigation techniques, and source provenance tracing.

Core Learning Objectives

1. **Semantic Search Application Architecture:** Design dual-pipeline architectures separating batch indexing/vectorization from real-time query retrieval APIs.
2. **Retrieval-Augmented Generation (RAG) Fundamentals:** Understand how vector databases serve as long-term factual memory while LLMs act as natural language reasoning engines.
3. **Grounded Prompt Engineering:** Construct structured system prompts and context injection templates that constrain model responses to retrieved source documents.
4. **Source Provenance & Citation Tracing:** Implement metadata tracking across the retrieval pipeline to enable verifiable, audit-compliant AI outputs.

Part 1: Semantic Search & RAG Architecture

1.1 The Anatomy of a Semantic Search Application

A production semantic search system consists of two distinct operational workflows:

1. **The Ingestion & Indexing Pipeline (Asynchronous / Batch):** Ingests raw enterprise source documents, parses text, splits text into chunks, generates vector embeddings, and stores both the original chunk text, vector embeddings, and structured metadata in the vector database.

2. **The Search & Generation API (Real-Time / Low Latency):** Accepts a user's natural language query, converts the query string into a dense vector embedding using the same model, executes a similarity search against the database, retrieves the top-k most relevant context chunks, and formats the output for display or LLM generation.

Table 1: Comparison of traditional keyword search vs. semantic search vs. retrieval-augmented generation (RAG).

System Paradigm	Query Processing Strategy	Response Type	Primary Advantage	Major Limitation
Traditional Keyword Search (BM25)	Matches exact tokens and terms in an inverted index	List of matching document links/rows	Fast, precise for exact SKU/ID lookups	Fails on synonyms, typos, and semantic intent
Semantic Search (Vector Search)	Calculates geometric distance between query and chunk vectors	Ranked list of relevant text chunks	Captures conceptual meaning without exact word match	Returns raw text snippets; does not synthesize answers
Retrieval-Augmented Generation (RAG)	Retrieves relevant chunks and passes them as context to an LLM	Synthesized, natural language answer grounded in data	Direct, actionable answers with source citations	Requires careful context window and prompt design

1.2 Why RAG is the Enterprise AI Paradigm

Fine-tuning a Large Language Model—updating its internal parameters on private company documents—is expensive, computationally intensive, and creates static knowledge snapshots that quickly become outdated. Furthermore, fine-tuned models can still hallucinate information and cannot enforce row-level access permissions.

RAG provides an architectural alternative that decouples **knowledge storage** from **language reasoning**:

- **Vector Store / Database = Memory (Factual Knowledge Base):** Updates in real-time as new documents, dispatches, or telemetry logs are ingested.
- **Large Language Model = Processing Engine (Reasoning & Synthesis):** Uses natural language skills to summarize, analyze, and format retrieved evidence without needing to memorize enterprise facts in its weights.

Table 2: Trade-offs between LLM Fine-Tuning and Retrieval-Augmented Generation (RAG).

Dimension	Fine-Tuning LLM Parameters	Retrieval-Augmented Generation (RAG)
Knowledge Currency	Static (Frozen at time of training)	Real-time (Instant access to new DB records)
Source Provenance	None (Model cannot cite specific source files)	Full (Retrieval returns specific doc_id and page numbers)
Hallucination Risk	High (Model generates answers from memory)	Low (Model is strictly grounded in retrieved context)
Access Control & Security	Difficult (Model knows all or nothing)	Native (Filter retrieval based on user roles and permissions)
Implementation Cost	High (GPU clusters, training runs)	Low (Standard database queries + local LLM inference)

Part 2: RAG Pipeline Mechanics & Grounded Prompting

2.1 The RAG Execution Lifecycle

When a user submits a query to a RAG-enabled enterprise application, the request moves through five sequential phases:

1. **Query Vectorization:** The query string Q is converted into a D-dimensional vector $\mathbf{v}_Q$ using the exact embedding model used during document indexing.
2. **Similarity Search & Filtering:** The system executes an Approximate Nearest Neighbor (ANN) query against the vector database, applying structured metadata filters (e.g., equipment_model = 'GIGA-9000') to return the top-k chunks.
3. **Context Assembly:** The raw text contents of the top-k chunks are concatenated into a formatted context block, preserving metadata attributes (such as doc_id, section, and timestamp).
4. **Grounded Prompt Construction:** The context block is merged into a specialized system prompt template that instructs the LLM to answer *only* using the provided context.
5. **LLM Generation & Provenance Attachment:** The LLM generates a response anchored to the retrieved context, appending citation markers corresponding to source document IDs.

2.2 Constructing Grounded System Prompts

The prompt sent to the LLM must explicitly constrain the model's behavior to eliminate hallucinations and prevent the LLM from relying on unverified external assumptions.

Essential System Prompt Directives

- **Strict Context Constraint:** Explicitly instruct the model that it must answer using **only** the facts provided in the [RETRIEVED CONTEXT] block.
- **Uncertainty Fallback:** Direct the model to respond with *"I do not have sufficient information in the provided records to answer this question"* if the retrieved context does not contain the answer.
- **Citation Mandate:** Require the model to reference source metadata IDs (e.g., [Doc: DOC-GIGA-901]) whenever asserting a technical fact.

Table 3: Structural components of an enterprise grounded prompt template.

Prompt Section	Purpose / Operational Directive	Example Template Format
System Role & Directive	Sets persona and strict grounding boundaries	You are an expert technical assistant for Project Gigavolt. Answer using ONLY the retrieved context.
Retrieved Context Block	Injects top-k retrieved chunks formatted with metadata	[CONTEXT]\nChunk 1 (Doc ID: DOC-101): Cell temp limits...\nChunk 2...
User Query	Formats the natural language question	[USER QUERY]\nWhat is the maximum safe operating temperature for battery module B?
Output Constraints	Dictates formatting, tone, and citations	If the answer is not in the context, state "Information unavailable". Include [Doc ID] citations.

Part 3: Advanced Design Patterns & Evaluation

3.1 Managing Context Window & Re-ranking

A common failure mode in RAG applications is **context stuffing**: sending too many retrieved chunks to the LLM. This can exceed the model's token limit, increase inference latency, and cause the LLM to overlook critical details located in the middle of long prompts (the "Lost in the Middle" phenomenon).

Advanced RAG pipelines introduce **re-ranking**:

1. **Initial Vector Retrieval:** Retrieve a broad candidate set (e.g., $k = 20$) using fast vector distance calculations.
2. **Cross-Encoder Re-ranking:** Pass the query and candidate chunks through a specialized cross-encoder model that scores token-level interactions, selecting the top $N = 3$ most relevant chunks.

3. **Prompt Injection:** Send only the top N re-ranked chunks to the LLM context window.

Table 4: Key performance indicators for evaluating RAG pipeline quality.

RAG Metric	Evaluation Question	Common Root Cause of Failure
Retrieval Recall	Did the vector search find the correct document chunks containing the answer?	Poor chunking strategy, small overlap, or embedding model misalignment
Faithfulness / Grounding	Is the LLM's answer completely supported by the retrieved context chunks?	Weak system prompt instructions; LLM relying on pre-trained memory
Answer Relevance	Does the generated response directly answer the user's specific query?	Irrelevant chunks injected into prompt or poor query vectorization
Provenance Accuracy	Do output citations accurately match the source document chunk metadata?	Missing metadata fields in context assembly step

Part 4: Practical Reference Script

The following complete Python script demonstrates an end-to-end RAG workflow: indexing document chunks with vector embeddings, processing a natural language user query, retrieving top context chunks using cosine distance, constructing a grounded prompt template, and executing local LLM generation (simulated or using Ollama/local inference API).

```
# ================================================================================

# ISTM 622: End-to-End Retrieval-Augmented Generation (RAG) Pipeline

# Focus: Vector Retrieval, Grounded Prompt Engineering, and Local LLM Synthesis

# Language: Python 3.10+

# Dependencies: sentence-transformers, numpy

# ================================================================================


import json

import numpy as np

from typing import List, Dict, Any

from sentence_transformers import SentenceTransformer
```

```
class RAGContextEngine:
    def __init__(self, embedding_model_name: str = "all-MiniLM-L6-v2"):
        """Initializes the vector embedding model for retrieval."""
        print(f"Initializing RAG Embedding Engine ({embedding_model_name})...")
        self.model = SentenceTransformer(embedding_model_name)
        self.vectors: np.ndarray = np.empty((0, 384))
        self.chunks: List[Dict[str, Any]] = []

    def index_chunks(self, document_chunks: List[Dict[str, Any]]) -> None:
        """Embeds and indexes document chunks into memory."""
        texts = [chunk["text"] for chunk in document_chunks]
        embeddings = self.model.encode(texts, convert_to_numpy=True)

        # Unit normalization for fast dot-product cosine similarity
        norms = np.linalg.norm(embeddings, axis=1, keepdims=True)
        normalized_embeddings = embeddings / norms

        self.vectors = normalized_embeddings
        self.chunks = document_chunks
        print(f"Indexed {len(document_chunks)} chunks for RAG retrieval.")

    def retrieve_context(self, query: str, top_k: int = 2) -> List[Dict[str, Any]]:
        """Retrieves top-k most semantically relevant chunks for a user query."""
        raw_query_vector = self.model.encode([query], convert_to_numpy=True)[0]
        query_vector = raw_query_vector / np.linalg.norm(raw_query_vector)

        # Compute Cosine Similarity via Dot Product
        scores = np.dot(self.vectors, query_vector)
```

```
        top_indices = np.argsort(scores)[::-1][:top_k]

        retrieved = []
        for idx in top_indices:
            chunk = self.chunks[idx].copy()
            chunk["score"] = float(scores[idx])
            retrieved.append(chunk)
        return retrieved

    def construct_grounded_prompt(self, query: str, context_chunks: List[Dict[str, Any]]) -> str:
        """Assembles context chunks into a strict, hallucination-resistant prompt."""
        context_str = ""
        for idx, chunk in enumerate(context_chunks, start=1):
            context_str += (
                f"--- CONTEXT BLOCK {idx} ---\n"
                f"Source Doc: {chunk['metadata']['doc_id']} ({chunk['metadata']['title']})\n"
                f"Section: {chunk['metadata']['section']}\n"
                f"Content: {chunk['text']}\n\n"
            )

        prompt_template = f"""SYSTEM DIRECTIVE:
You are an authoritative technical assistant for Project Gigavolt.
You must answer the USER QUERY using ONLY the information provided in the RETRIEVED CONTEXT below.
If the answer cannot be determined from the context, state: "I do not have sufficient information in the grounded records to answer this query."
Do NOT use external assumptions or unverified pre-trained memory. Always cite the Source Doc ID.

[RETRIEVED CONTEXT]
{context_str.strip()}
```

```
[USER QUERY]
{query}

[GROUNDED ANSWER]:"""
        return prompt_template

    def generate_simulated_llm_response(self, prompt: str, context_chunks: List[Dict[str, Any]]) -> str:
        """Simulates grounded generation based on prompt instructions (or connects to local Ollama API)."""
        # In production: pass `prompt` to local LLM via Ollama (e.g., Llama-3-8B)
        # Here we demonstrate the structured grounded output expected from the LLM
        sources = ", ".join(set([f"[{c['metadata']['doc_id']}]" for c in context_chunks]))
        return (
            f"Based on technical specifications {sources}, the maximum allowable cell temperature "
            f"before initiating thermal runaway protocols is 65 degrees Celsius. If exceeded, technicians "
            f"must immediately disconnect primary inverter feed lines and engage coolant loop bypass switches."
        )


# --- Execution Walkthrough ---
if __name__ == "__main__":
    # 1. Sample Structured Document Chunks
    sample_corpus = [
        {
            "chunk_id": "CHK-901",
            "text": "Cell temperatures exceeding 65C indicate severe thermal runaway risk. Immediately disconnect inverter lines.",
            "metadata": {"doc_id": "SOP-BATT-01", "title": "Battery Thermal SOP", "section": "Emergency Protocols"}
        },
        {
```

```
        "chunk_id": "CHK-902",
        "text": "Routine maintenance requires re-torquing coolant loop M8 bolts to 25 Nm every 12 months.",
        "metadata": {"doc_id": "MAN-MAINT-04", "title": "Gigavolt Maintenance Manual", "section": "Mechanical Torque"}
    },
    {
        "chunk_id": "CHK-903",
        "text": "Warranty claim approvals require verified dispatch logs signed by a certified technician.",
        "metadata": {"doc_id": "POL-CLAIM-02", "title": "Warranty Policy", "section": "Claim Requirements"}
    }
]

# 2. Instantiate and Index
engine = RAGContextEngine()
engine.index_chunks(sample_corpus)

# 3. Process Query
user_query = "What is the cell temperature threshold for thermal runaway and what action is required?"
print(f"\nUser Query: '{user_query}'")

# 4. Retrieve Context
relevant_chunks = engine.retrieve_context(user_query, top_k=1)
print(f"\nRetrieved {len(relevant_chunks)} grounding chunk(s). Top Score: {relevant_chunks[0]['score']:.4f}")

# 5. Build Grounded Prompt
grounded_prompt = engine.construct_grounded_prompt(user_query, relevant_chunks)
print("\n" + "="*20 + " GENERATED GROUNDED PROMPT " + "="*20)
print(grounded_prompt)
```

```
# 6. Generate Response
response = engine.generate_simulated_llm_response(grounded_prompt, relevant_chunks)
print("\n" + "="*20 + " LLM GENERATED ANSWER " + "="*20)
print(response)
```

Review & Preparation Questions

1. **Architecture:** Explain why Retrieval-Augmented Generation (RAG) is preferred over fine-tuning LLM model parameters for enterprise applications with frequently updating data.
2. **Prompt Engineering:** How does including an explicit uncertainty fallback instruction (e.g., *"If information is missing, state 'Insufficient data'"*) reduce hallucination in RAG pipelines?
3. **Context Window Management:** What is the "Lost in the Middle" phenomenon in LLM context windows, and how does re-ranking help resolve it?
4. **Source Provenance:** Why is attaching structured metadata (doc_id, page_number) to context chunks necessary for meeting enterprise audit and compliance standards?

References & Further Reading

- **Lewis, P., et al. (2020).** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS 2020)*.
- **MariaDB Corporation.** "Building AI Applications with MariaDB Vector & RAG." *MariaDB Knowledge Base*, 2026.
- **Liu, N. F., et al. (2023).** "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*.
- **Ollama Team.** "Running Local Language Models for Enterprise RAG." *Ollama Documentation*, 2026.